

MechMem-RTL: Reusing Verified Mechanism Memories for LLM-Based RTL Repair

Mingyu Cheng, Junjie Gao, Jinhua Cui, Kuncai Zhong*

College of Semiconductors, Hunan University, China

Email: {mycheng, junjie1, jhcui, kczhong}@hnu.edu.cn; *Corresponding author

Abstract

Large language models (LLMs) can automatically repair register-transfer-level (RTL) designs. However, fixing complex sequential logic errors requires reusing past debugging experience. Existing retrieval-augmented generation (RAG) relies on task-text similarity to provide this experience. This text-based approach often misguides the model because natural language poorly reflects cycle-level hardware execution semantics. To address this, we present MechMem-RTL, a repair framework that reuses verifier-confirmed repair records instead of text similarity. Each stored record strictly links trigger evidence, a diagnosed failure mechanism, a local repair action, preservation constraints, and a verification summary. For a new failure, MechMem-RTL injects a past record only when deterministic verifier evidence is strictly compatible with the stored trigger. Otherwise, the system uses only current verifier evidence. We evaluate MechMem-RTL on 48 public sequential RTL tasks across six repair models. With at most two repair attempts per task, MechMem-RTL successfully resolves 180 out of 288 task-model pairs, outperforming standard feedback repair (109 pairs) and task-similarity RAG (107 pairs).

Keywords

Automated RTL repair, Large language models, Retrieval-augmented generation, Verification feedback, Sequential logic.

1 Introduction

Large language models (LLMs) have emerged as practical tools for register-transfer-level (RTL) design [1–4]. However, because their initial outputs rarely pass all verification checks on the first attempt, automated RTL workflows rely heavily on an iterative repair loop. Within this loop, the model proposes localized revisions based on compiler or simulator feedback. These revisions are highly constrained, as a valid repair must strictly preserve the original module interface and match cycle-accurate behavior under the target testbench.

Satisfying these cycle-accurate constraints makes fixing sequential logic particularly difficult. A single functional mismatch can stem from flawed finite-state machine (FSM) transitions, incorrect event priorities, or misaligned protocol boundaries. Current verifiers usually report the immediate failing symptom, such as a signal mismatch at a specific clock cycle. They do not identify the structural root cause. Since LLMs do not directly model hardware semantics, they struggle to map these surface symptoms to the correct code edit. Closing this gap requires the repair system to reuse prior debugging experience rather than relying solely on the latest error message.

A common approach to providing this past experience is retrieval-augmented generation (RAG) based on task-text similarity [5, 6]. However, in sequential RTL debugging, textual similarity is a weak proxy for hardware execution behavior. Tasks with completely different descriptions can fail due to the same state-update error. Conversely, similar task descriptions can fail for entirely different cycle-level reasons. Therefore, reusing repair experience based merely on text similarity often misguides the LLM.

To address this limitation, we present MechMem-RTL, an automated framework that extracts and reuses verifier-confirmed repair mechanisms. Upon a successful repair, the system constructs a structured mechanism memory record. Each record explicitly links trigger evidence, a diagnosed failure mechanism, a localized repair action, preservation constraints, and a verification summary. For subsequent repairs, a deterministic evidence filter injects a stored record into the prompt only when the current compiler or simulator diagnostics are strictly compatible with the stored trigger. Otherwise, the context is constructed from current verifier evidence.

Specifically, we make the following contributions.

- We introduce a verifier-confirmed mechanism memory designed for sequential RTL repair. It formally encapsulates five key elements: verification trigger evidence, a diagnosed failure mechanism, a targeted repair action, behavior preservation constraints, and a verification summary (see Section 3).
- We design a hardware-aware deterministic evidence filter. It selects past records using strict compiler and simulator diagnostics rather than task-text similarity or language model heuristics, ensuring high-fidelity guidance (see Section 3).
- We curate SeqRTL, an evaluation set of 48 public sequential RTL tasks with failing initial designs. We thoroughly compare our approach against standard feedback repair and task-similarity RAG across six different repair models (see Sections 4 and 5).

Experimental results confirm the effectiveness of MechMem-RTL. Given a strict limit of two repair attempts per task, our method successfully resolves 180 out of 288 task-model pairs. Under identical conditions, standard feedback repair and task-similarity RAG resolve only 109 and 107 pairs, respectively. We will make our framework code and evaluation artifacts publicly available.

2 Background and Motivation

In this section, we explain why automated RTL repair requires reusable knowledge grounded in verifier evidence rather than task text.

RTL generation and evaluation. Recent studies establish a standard paradigm for LLM-aided RTL design using executable

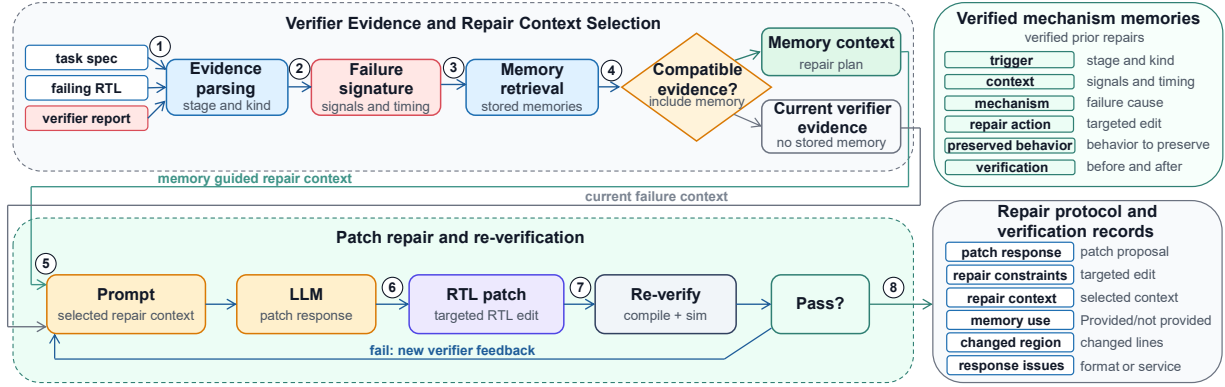


Figure 1: Workflow of MechMem-RTL. Verifier evidence determines whether a stored repair record guides the next repair.

checks [1, 2, 7–10]. In this setting, a model proposes an RTL candidate, and an automated harness verifies its functional correctness. Although domain-specific models trained on RTL data improve initial generation quality [3, 4], complex designs rarely pass all checks immediately. Because many candidates fail on the first attempt, the design workflow naturally enters a repair stage.

RTL repair and failure mechanisms. Current RTL repair systems guide LLM edits using compiler errors or simulator feedback [11–14], often validated against executable testbenches [15, 16]. However, verifier feedback typically reports a surface symptom rather than the structural root cause. For example, a simulator may flag an output mismatch at a specific clock cycle, while the actual fix requires changing an FSM transition, an event priority rule, or a protocol boundary. Recent error analyses confirm that effective repair must distinguish specific failure mechanisms rather than just binary pass or fail outcomes [17]. This gap between immediate symptoms and structural root causes makes it necessary to reuse prior debugging experience.

Limitations of task-similarity RAG. A common approach to reusing past experience is task-similarity RAG [5, 6]. For sequential RTL debugging, however, text similarity is a weak proxy for hardware execution behavior. Tasks with completely different descriptions can fail due to the same state-update error, while similar descriptions can fail for entirely different cycle-level reasons. Retrieval based solely on task wording often introduces context that looks relevant but mismatches the current failure mechanism. Therefore, the repair context should be selected using evidence tied directly to compiler diagnostics, timing, and signal behavior.

Reusable verification knowledge. Executable RTL checks naturally provide this deterministic evidence. Because RTL repair is strictly constrained by module interfaces and cycle-accurate behavior, a successful repair is much more than a simple text solution. It is a verifiable debugging episode. General software repair often relies on execution traces or case-based reasoning [18–24]. In contrast, sequential RTL debugging demands reusable artifacts strictly tied to hardware timing and signal evidence [25]. MechMem-RTL applies this principle by capturing each successful repair as a verifier-confirmed mechanism memory record. Each record links the failure

trigger, the diagnosed mechanism, the local repair action, preservation constraints, and a verification summary. We formalize the structure and filtering of these mechanism memories in the next section.

3 Proposed Method

In this section, we formalize the MechMem-RTL framework. We first detail how the system creates and reuses verifier-confirmed repair records. We then formally define the memory record structure, explain the deterministic evidence filter that controls reuse, and describe the final patch-level diagnostics.

3.1 Core Workflow

Figure 1 illustrates the core workflow of MechMem-RTL. The system operates in two distinct phases. These are memory creation and memory-guided repair.

During memory creation, the system extracts knowledge from successful debugging episodes. A memory record is stored only when a proposed RTL patch passes the executable testbench. This strict rule keeps the memory bank anchored in verified hardware evidence rather than unverified model hypotheses.

During memory-guided repair, the inputs consist of the task specification, the current faulty RTL candidate, and the latest verifier report. MechMem-RTL normalizes this verifier evidence and retrieves initial candidate records from the memory bank. A deterministic evidence filter then evaluates whether any retrieved record is strictly compatible with the current failure. If a compatible record survives this filter, it is injected into the prompt as the selected repair context. If no compatible memory record is selected, the repair context is constructed from the current verifier evidence without reusing stored memory. The model then proposes a localized patch, and the verifier evaluates the modified RTL. If the candidate still fails, the process repeats using updated verifier feedback until it reaches the predefined iteration limit.

Crucially, this workflow explicitly targets sequential RTL failures involving control and timing behavior. Such failures include finite-state machine transitions, output timing delays, protocol boundaries, counter thresholds, pulse alignments, reset logic, and state-history updates. Compile and tool-compatibility errors are

also addressed when they surface within these repair loops. This bounded scope ensures that memory reuse is driven entirely by cycle-level hardware semantics.

3.2 Mechanism Memory Record

At the core of this workflow is the mechanism memory, a structured diagnostic record backed by executable verification. We formalize it as a five-element tuple $m = (T, M, R, P, V)$. Here, T represents the trigger evidence. M is the diagnosed failure mechanism. R specifies the localized repair action. P outlines the behavior preservation constraints. Finally, V provides the verification summary.

The trigger T captures normalized compiler or simulator diagnostics. It includes the verification stage, failure kind, affected outputs, and first mismatch time if available. The mechanism M summarizes the hardware failure cause, such as a state-output timing delay or a serial bit-order inversion. The repair action R records the accepted localized code edit. The preservation field P identifies surrounding logic, interface constraints, or timing behavior that must remain unchanged. The verification summary V simply logs the transition from a failing outcome to a passing one.

While the mechanism (M) and preservation (P) fields are generated during the repair attempt, they are never stored in isolation. MechMem-RTL commits the complete five-element record to the memory bank only after the associated patch passes the executable testbench. This verification-gated process ensures that the reusable rationale is tied to a concrete and verifiable edit.

Table 1: Example verifier-confirmed repair record.

Field	Stored content
Prior task	VerilogEval shift-count task
Trigger evidence	Output q: 1886/2071 mismatched samples; first mismatch at time 10
Mechanism	Wrong direction in MSB-first serial shift
Repair action	Replace {data, q[3:1]} with {q[2:0], data}
Preserved behavior	Preserve counter behavior, interface, and hold behavior
Verification	Same testbench: mismatch before repair, pass after repair

Table 1 provides an example of a verifier-confirmed repair record. The trigger and verification fields anchor the memory in objective hardware evidence. Meanwhile, the mechanism, repair action, and preservation fields specify exactly what knowledge can be reused. Consequently, the reusable unit is not a mere copy of a previous solution. Instead, it serves as a focused repair context centered on one specific failure mechanism.

3.3 Evidence Filtering

With the memory bank populated by these structured records, the next critical challenge is accurately identifying the correct repair context for a newly encountered failure. As illustrated in Figure 2, this selection process operates in two stages. First, a retrieval module proposes initial candidate records based on broad normalized verifier evidence. Second, the deterministic evidence filter rigorously screens these candidates to guarantee strict hardware compatibility with the current failure.

Instead of relying on probabilistic language model judgments, the filter extracts deterministic hardware features to perform these

strict compatibility checks. These features correspond to the parallel comparison lanes in Figure 2, mapping the current failure evidence directly to the stored trigger T . The filter systematically evaluates the verification stage, failure kind, affected output signals, and temporal diagnostic signatures. To prevent data leakage, it utilizes task metadata to exclude records originating from the same task or known duplicate tasks.

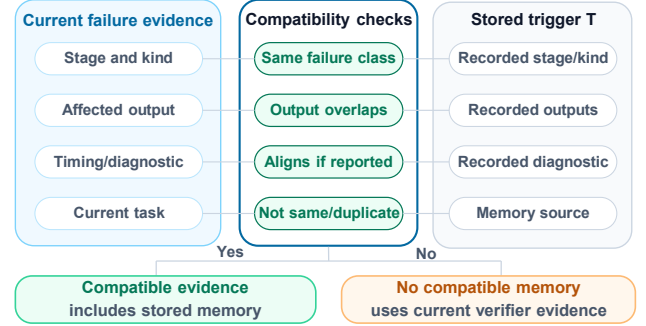


Figure 2: Deterministic evidence filtering. Matched evidence exposes memory; otherwise, repair uses current evidence.

Compatibility requires a strict set of mechanism-specific constraints. For example, a compile-stage memory can only be reused for a compatible compile-stage diagnostic. A one-cycle output-timing memory requires a matching simulation mismatch pattern on a related output port or timing phase. Similarly, a serial bit-order memory requires matching signal and phase evidence. These deterministic constraints are much stricter than natural language text similarity. They prevent weakly related hardware contexts from bypassing the filter and misleading the repair model.

After filtering, MechMem-RTL constructs the final repair context. If compatible memories survive, the prompt includes the surviving records and requests a targeted patch. If no compatible memories survive, the retrieved records are withheld from the model, and the repair context is constructed from the current verifier evidence without reusing stored memory. Section 5 reports these context-selection decisions as explicit repair diagnostics.

3.4 Repair Patches and Diagnostics

Having determined the appropriate repair context, the framework constructs the final prompt. This prompt integrates the current faulty RTL, the immediate verifier feedback, and any memory records that survive the filter. Regardless of the specific context type, the system explicitly instructs the model to diagnose the failure mechanism and return a localized patch. Furthermore, the prompt requires the model to identify the modified code region and specify which surrounding behaviors must remain functionally intact. The expected response is a targeted edit rather than a full-module rewrite.

Once the language model generates this candidate patch, the system must enforce strict hardware boundaries. The repaired RTL must strictly preserve the original module interface and keep logic modifications strictly confined to the diagnosed failure zone. To monitor adherence to these structural constraints, the framework

continuously logs key process metrics. These tracked diagnostics include response validity, repair-constraint violations, full-module rewrite attempts, line modification counts, and memory filtering statistics. While these metrics do not replace the primary verified pass rate, they provide essential transparency. Ultimately, this tracking ensures that the automated debugging process strictly adheres to practical hardware engineering standards.

4 Experimental Setup

To evaluate the proposed framework, we establish a strictly matched repair protocol. By fixing the task set, initial failing designs, iteration limits, and verification harness across all methods, we ensure a fair and fully reproducible comparison [1, 2, 15, 26]. Figure 3 outlines this shared evaluation pipeline.

4.1 SeqRTL Evaluation Set

Because our focus is automated debugging rather than initial generation, we construct SeqRTL, a dedicated evaluation set of 48 public sequential RTL tasks. To guarantee a zero initial pass rate, each task begins with a recorded faulty design originally generated by GPT-5.2. We include a task only if it features sequential control logic, has public provenance, and includes a verified reference design.

To prevent data leakage, we rigorously exclude any tasks overlapping with the source pool used to build our mechanism memory bank. We also filter out known duplicates, initially passing designs, and tasks without runnable testbenches. The final 48 tasks and their starting failures remain strictly frozen throughout the evaluation. These tasks originate from established benchmarks (e.g., HDLBits, CodeV-R1, RTL2M) [2, 15] and cover diverse hardware behaviors, including protocol handshaking, reset initialization, edge capture, counter thresholds, and state-history updates.

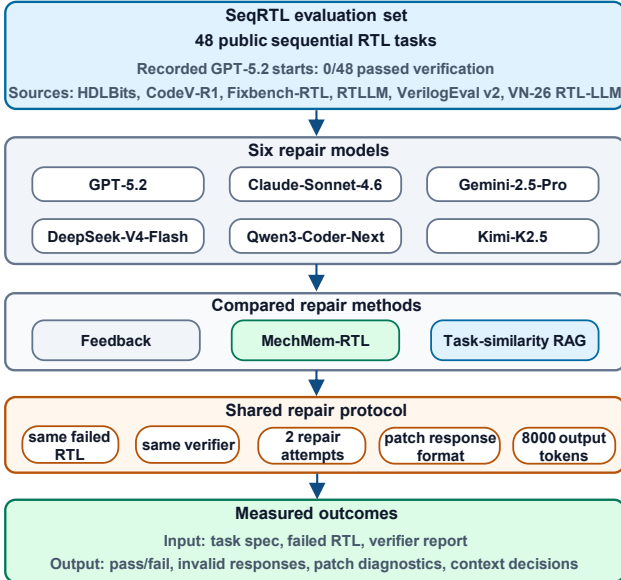


Figure 3: Matched repair protocol for SeqRTL across three repair methods and six repair models.

4.2 Repair Methods

We compare three repair methods under a matched design. All methods share the same task specification, current faulty RTL candidate, and verifier feedback. The only difference is the additional repair context.

Standard feedback repair. This baseline relies exclusively on the current faulty RTL candidate and the immediate compiler or simulator diagnostic report.

Task-similarity RAG. This baseline represents the conventional textual retrieval approach. It retrieves up to three solved task examples based on lexical overlap across task titles, specifications, and module interfaces [5, 6]. To construct the prompt, these retrieved examples provide task descriptions, reference interfaces, and retrieval metadata. The target task and its known duplicates are strictly excluded from the retrieval pool to ensure fairness.

MechMem-RTL. Our proposed method utilizes a separate bank of 53 verifier-confirmed mechanism memory records, prepared independently from the evaluation runs. To maintain the integrity of this bank, a source repair episode contributes a record only when its patch successfully passes verification. During the evaluation runs, the system injects a stored record into the prompt only if the current diagnostic report is strictly compatible with the stored trigger evidence. If no compatible memory record is selected, the repair context is constructed from the current verifier evidence without reusing stored memory.

To ensure strict fairness, all methods repair the same 48 starting designs. Each method receives a maximum of two repair attempts per task and an 8000-token output limit. All methods use identical patch response formats and share the same retry rules for handling invalid formats. Importantly, models are explicitly constrained to preserve the original module interface, avoid full-module rewrites, and restrict edits strictly to the diagnosed failure zone.

Every repair attempt utilizes the identical Icarus Verilog simulation harness. The models receive text-based compiler diagnostics and testbench mismatch summaries, but no raw Value Change Dump (VCD) traces or formal counterexamples. We evaluate these methods across six repair models (GPT-5.2, Claude-Sonnet-4.6, Gemini-2.5-Pro, DeepSeek-V4-Flash, Qwen3-Coder-Next, and Kimi-K2.5), yielding a total of 864 matched evaluation runs.

4.3 Metrics

The primary evaluation metric is the verified repair pass rate, which measures the fraction of initially failing designs that successfully pass the verification harness within the allowed two attempts. We report the absolute repair gains of MechMem-RTL compared to both baselines across all six models.

To further interpret the final outcomes, we monitor patch-contract diagnostics. As established in Section 3.4, these track invalid response formats, line modification counts, full-module rewrite attempts, and repair-constraint violations. While these diagnostics do not replace the primary pass rate, they provide essential transparency into system stability and verify that the models adhere to practical hardware constraints under identical testing conditions.

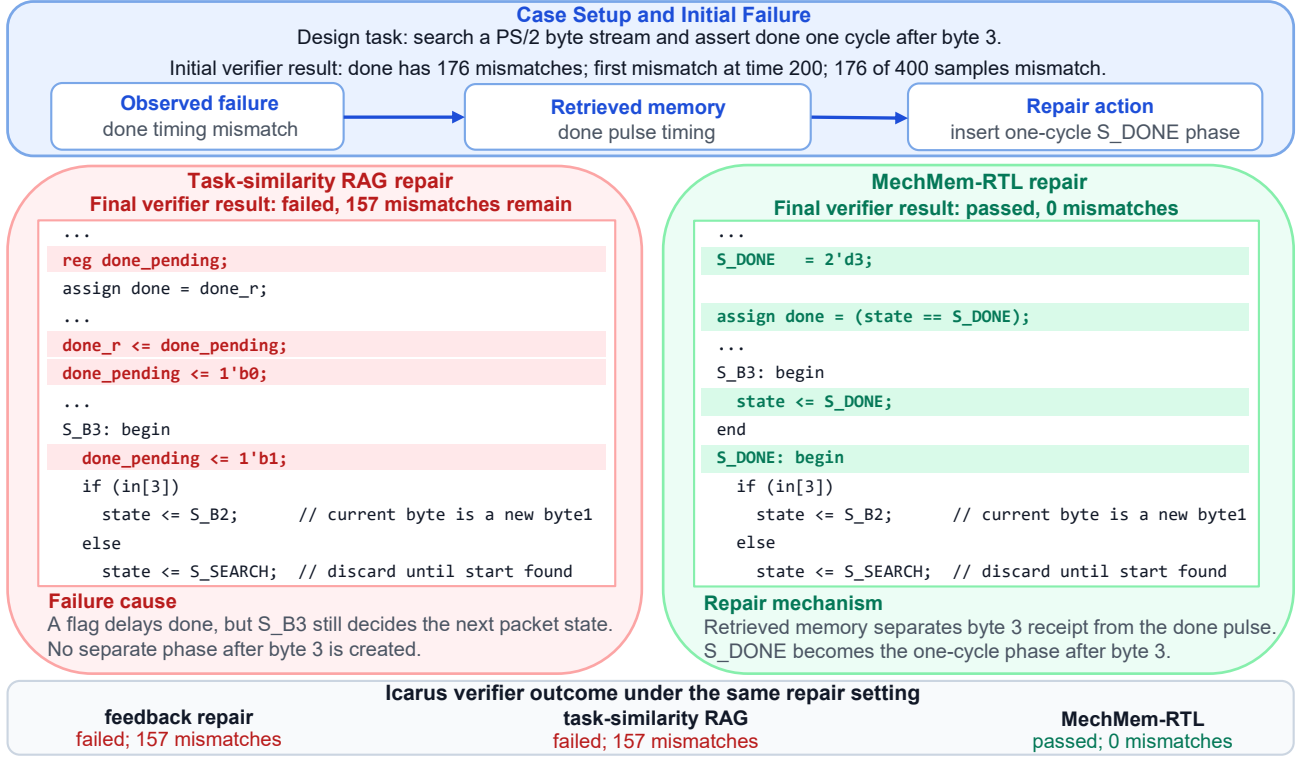


Figure 4: PS/2 packet-boundary case study with a separate one-cycle done phase.

5 Results and Discussion

This section evaluates repair performance across 864 matched runs on SeqRTL. As established in Section 4, all comparisons share identical starting designs, verifiers, repair limits, and token budgets. Invalid repair responses are strictly counted as failed attempts. Unless stated otherwise, all counts refer to task-model pairs. Table 2 reports the aggregate repair outcomes, and Figure 4 details a specific packet-boundary repair case.

Table 2: Verified repair counts under the matched protocol. Gain is over the stronger baseline for each repair model.

Repair model	Feedback	Task-sim. RAG	MechMem-RTL	Gain
GPT-5.2	22	23	36	+13
Claude-Sonnet-4.6	23	21	33	+10
Gemini-2.5-Pro	19	19	26	+7
DeepSeek-V4-Flash	25	22	29	+4
Qwen3-Coder-Next	8	5	23	+15
Kimi-K2.5	12	17	33	+16
Total	109/288	107/288	180/288	+65

5.1 Matched Repair Results

Table 2 summarizes the verified repairs under the shared protocol. MechMem-RTL achieves the highest verified repair count across all six repair models, successfully resolving 180 out of 288 task-model

pairs. In comparison, standard feedback repair solves 109 pairs, and task-similarity RAG solves 107 pairs. This yields an aggregate gain of 65 repairs over the stronger baselines in the six per-model comparisons.

Notably, task-similarity RAG fails to improve upon standard feedback repair in aggregate. This validates our core motivation: natural language task text alone is an unreliable proxy for sequential hardware behavior. Retrieved examples often match at the specification level but fundamentally mismatch the current cycle-level failure, thereby misleading the model [27, 28].

Figure 4 illustrates this specific failure mode on a PS/2 packet-boundary task. Starting from the same faulty design, task-similarity RAG merely attempts to delay a done flag, leaving 157 simulation mismatches unresolved. In contrast, MechMem-RTL retrieves a compatible pulse-timing memory. This deterministic context guides the model to structurally introduce a separate one-cycle S_DONE phase, completely resolving the mismatch.

Figure 5 categorizes these verified repairs by mechanism family. The most substantial gains emerge in FSM timing, protocol-boundary, and serial-shift categories. These failures demand precise adjustments to cycle-level timing and state transitions, confirming that verifier-confirmed memories provide critical structural context. Compile/tool and counter/timer families also exhibit consistent gains. Performance on state-update pairs remains tied. Interestingly, standard feedback repair resolves slightly more edge or sticky-capture pairs. Such localized capture failures typically expose

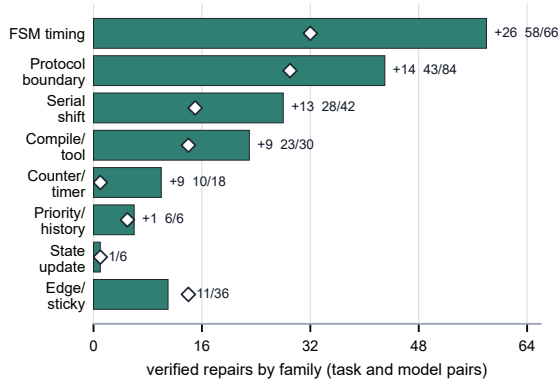


Figure 5: Verified repairs categorized by mechanism family. Bars represent MechMem-RTL while diamonds indicate the stronger baseline.

sufficient diagnostic information in the immediate verifier report, making historical context less vital. This family-level breakdown comprehensively clarifies the source of our aggregate gains.

5.2 Paired Outcomes

To confirm that the aggregate gain represents a true capability expansion rather than a simple shift in solved tasks, Figure 6 details the uniquely resolved pairs. Against standard feedback repair, MechMem-RTL uniquely resolves 87 complex failures, while missing only 16 pairs handled by the baseline. Against task-similarity RAG, the corresponding unique counts are 86 pairs for MechMem-RTL and only 13 pairs for the RAG baseline. These paired outcomes demonstrate that MechMem-RTL consistently resolves complex structural failures that remain out of reach for both baselines.

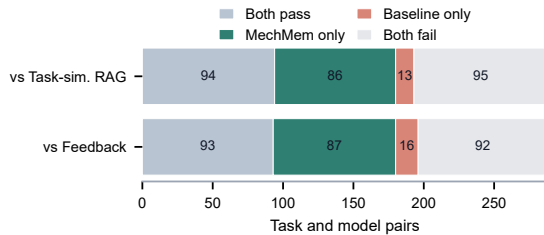


Figure 6: Paired outcomes over 288 task-model pairs. Each bar compares MechMem-RTL with one baseline.

5.3 Repair Diagnostics

To maintain end-to-end transparency under the shared protocol, we track response format failures and count invalid outputs as failed attempts [21, 22]. Format failures are unevenly distributed. Gemini-2.5-Pro produces 61 invalid responses (mostly empty), and DeepSeek-V4-Flash produces 21 failures (often due to token limits). Other models produce at most 8 invalid responses. These metrics

reveal how different model architectures interact with identical structural repair contexts.

Patch-contract diagnostics further reveal the nature of the generated fixes. On average, MechMem-RTL modifies 7.54 lines per task-model pair, slightly higher than the 6.38 and 6.55 lines modified by the two baselines. In sequential logic, this slight increase suggests that MechMem-RTL performs necessary structural refactoring rather than trivial signal toggling. Importantly, this structural awareness does not degrade overall stability: full-module rewrite attempts remain extremely rare (3 for MechMem-RTL, 4 for each baseline). Furthermore, MechMem-RTL triggers fewer repair-constraint violations than both baselines, confirming its strict adherence to hardware boundaries.

5.4 Use of Stored Memories

A critical design principle of MechMem-RTL is to reuse stored memory only when the current hardware evidence is strictly compatible with the stored trigger. Table 3 tracks these deterministic routing decisions.

Table 3: Context selection and outcomes for MechMem-RTL.

Repair context	Pairs	Pass	Rate
With stored memory	162	118	72.8%
Without stored memory	126	62	49.2%
Total	288	180	62.5%

Across the six repair models, 162 task-model pairs receive a prompt augmented with stored memory, yielding 118 verified passes. The remaining 126 task-model pairs use repair contexts constructed from the current verifier evidence without stored memory, yielding 62 verified passes. This selective routing shows that MechMem-RTL uses historical context only when the current evidence is compatible with a stored trigger, reducing the risk of misleading the repair model with irrelevant context.

Overall, MechMem-RTL achieves the highest verified repair count across all six repair models under the matched protocol. These results support the central claim that sequential RTL repair benefits from evidence-based context selection rather than task-text similarity alone.

6 Conclusion

This paper presents MechMem-RTL, a framework for reusing verifier-confirmed experience in automated RTL repair. Instead of relying on unreliable text similarity, the system stores successful debugging episodes as structured mechanism memories. It reuses a memory only when the current verifier evidence is strictly compatible with the stored trigger. Across 48 public sequential RTL tasks and six repair models, MechMem-RTL successfully resolves 180 out of 288 task-model pairs, outperforming standard feedback repair (109) and task-similarity RAG (107). These results demonstrate that sequential RTL repair benefits from selective memory reuse grounded in deterministic verification evidence. This approach is particularly effective for resolving complex cycle-level functional errors, where task text alone provides weak and misleading guidance.

References

- [1] M. Liu et al. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *ICCAD*. 1–8.
- [2] Y. Lu et al. 2024. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *ASP-DAC*. 722–727.
- [3] S. Thakur et al. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM TODAES* 29, 3 (2024), 46:1–46:31.
- [4] S. Liu et al. 2025. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE TCAD* 44, 4 (2025), 1448–1461.
- [5] P. Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*. 9459–9474.
- [6] J. Chen et al. 2024. Benchmarking Large Language Models in Retrieval-Augmented Generation. In *AAAI*. 17754–17762.
- [7] C. Li et al. 2025. AutoSilicon: Scaling Up RTL Design Generation Capability of Large Language Models. *ACM TODAES* 30, 6 (2025), 97:1–97:21.
- [8] Y. Wang et al. 2026. VeriAgent: A Tool-Integrated Multi-Agent System with Evolving Memory for PPA-Aware RTL Code Generation. arXiv:2603.17613.
- [9] N. R. Pinckney et al. 2025. Revisiting VerilogEval: A Year of Improvements in Large-Language Models for Hardware Code Generation. *ACM TODAES* 30, 6 (2025), 91:1–91:20.
- [10] Z. Fang et al. 2025. RTL Bench: A Multi-Dimensional Benchmark Suite for Evaluating LLM-Generated RTL Code. In *ICCD*. 566–573.
- [11] Y.-D. Tsai, M. Liu, and H. Ren. 2024. RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Model. In *DAC*. 53:1–53:6.
- [12] K. Xu et al. 2024. MEIC: Re-thinking RTL Debug Automation using LLMs. In *ICCAD*. 100:1–100:9.
- [13] X. Yao et al. 2025. HDLdebugger: Streamlining HDL debugging with Large Language Models. *ACM TODAES* 30, 6 (2025), 102:1–102:26.
- [14] A. Elnaggar and B. Tan. 2025. Adding Context to LLM-Guided Verilog Repair. In *ISQED*. 1–7.
- [15] S. Li et al. 2025. Fixbench-RTL: A Comprehensive Benchmark for Evaluating LLMs on RTL Debugging. In *AsianHOST*. 1–6.
- [16] Y. Tan et al. 2026. AutoVeriFix: Automatically Correcting Errors and Enhancing Functional Correctness in LLM-Generated Verilog Code. In *ASP-DAC*. 526–532.
- [17] J. Zhang et al. 2026. Understanding and Mitigating Errors of LLM-Generated RTL Code. *IEEE TCAD* (2026), 1–1.
- [18] A. Aamodt and E. Plaza. 1994. Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches. *AI Communications* 7, 1 (1994), 39–59.
- [19] C. S. Xia, Y. Wei, and L. Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *ICSE*. 1482–1494.
- [20] N. Jiang et al. 2023. Impact of Code Language Models on Automated Program Repair. In *ICSE*. 1430–1442.
- [21] C. S. Xia and L. Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT. In *ISSTA*. 819–831.
- [22] X. Yin et al. 2024. ThinkRepair: Self-Directed Automated Program Repair. In *ISSTA*. 1274–1286.
- [23] F. Li et al. 2025. Hybrid Automated Program Repair by Combining Large Language Models and Program Analysis. *ACM TOSEM* 34, 7 (2025), 1–28.
- [24] F. Mu et al. 2026. ExpeRepair: Dual-Memory Enhanced LLM-based Repository-Level Program Repair. arXiv:2506.10484. Accepted by FSE 2026.
- [25] Y. Hu et al. 2025. UVLLM: An Automated Universal RTL Verification Framework using LLMs. In *DAC*. 1–7.
- [26] C. E. Jimenez et al. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *ICLR*. 51 pages.
- [27] F. Shi et al. 2023. Large Language Models Can Be Easily Distracted by Irrelevant Context. In *ICML*. 31210–31227.
- [28] N. F. Liu et al. 2024. Lost in the Middle: How Language Models Use Long Contexts. *TACL* 12 (2024), 157–173.